

Exp No.: 6 Implementing Fabric Channels for Privacy

Aim

To create and configure multiple channels in a Hyperledger Fabric network, deploy chaincode on a specific channel, and demonstrate how Fabric channels provide data privacy and isolation between organizations.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Theory

A Fabric Channel is a private communication mechanism that allows a subset of organizations to maintain a separate ledger and execute transactions confidentially. Each channel has its own blockchain ledger, smart contracts (chaincode), and world state database. Organizations that are not members of a channel cannot access its ledger or transaction data. Channels enable enterprise applications to maintain confidentiality while sharing the same Hyperledger Fabric network.

Procedure

Step 1: Navigate to the Fabric Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

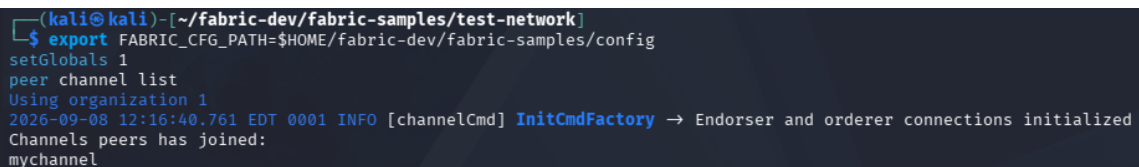
Step 2: Start the Hyperledger Fabric Network

```
./network.sh up createChannel -ca
```

Step 3: Verify the Existing Channel

List the channels joined by the peer.

```
peer channel list
```



```
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config
setGlobals 1
peer channel list
Using organization 1
2026-09-08 12:16:40.761 EDT 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
Channels peers has joined:
mychannel
```

Figure 1: Verifying peer has joined mychannel

Step 4: Create a New Private Channel

Create a second channel named privatechannel.

```
./network.sh createChannel -c privatechannel
```

Step 5: Deploy Chaincode on the Private Channel

Deploy the JavaScript Asset Transfer chaincode to privatechannel.

```
./network.sh deployCC -c privatechannel \
  -ccn basic -ccl javascript \
  -ccp ../asset-transfer-basic/chaincode-javascript
```

Set anchor peers for both organizations on the privatechannel to enable cross-org communication.

Figure 2: Anchor peer configuration and channel update for privatechannel

Figure 2: Anchor peer configuration and channel update for privatechannel

Step 7: Initialize the Ledger on the Private Channel

```
peer chaincode invoke ... -C privatechannel -n basic -c '{"function":"InitLedger","Args":[]}'
```

Step 8: Create an Asset on mychannel

```
peer chaincode invoke ... -C mychannel -n basic -c '{"function":"CreateAsset","Args":["channelAsset1","Blue","10","Arun","1000"]}'
```

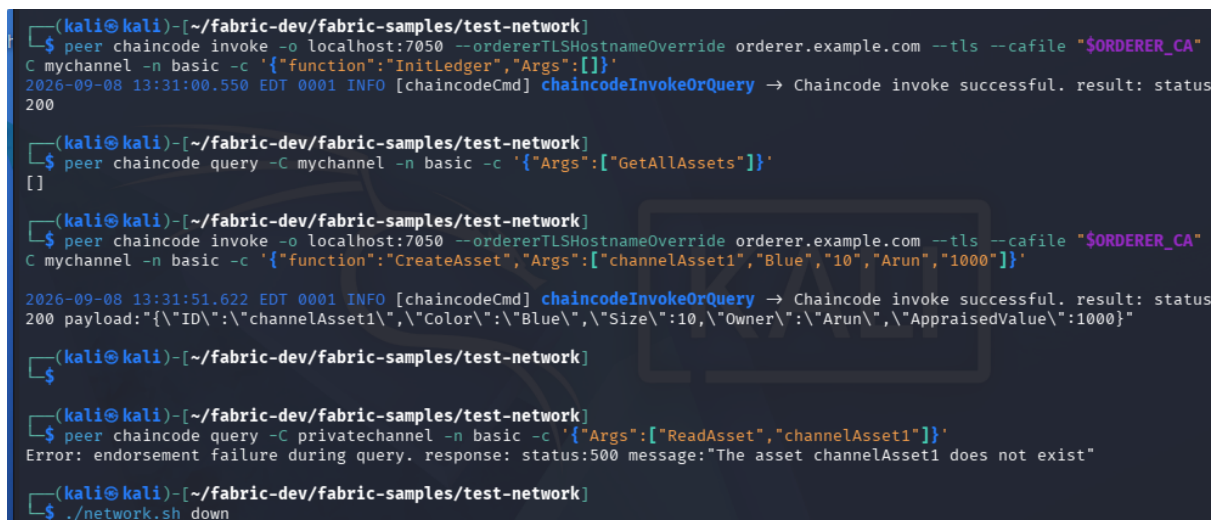
Step 9: Verify Channel Isolation

Attempt to read channelAsset1 from privatechannel — the asset should not exist, proving that data on one channel is isolated from the other.

```
peer chaincode query -C privatechannel -n basic -c '{"Args":["ReadAsset","channelAsset1"]}'
```

Step 10: Stop the Fabric Network

```
./network.sh down
```



```
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c '{"function":"InitLedger","Args":[]}'
2026-09-08 13:31:00.550 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status 200

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[]

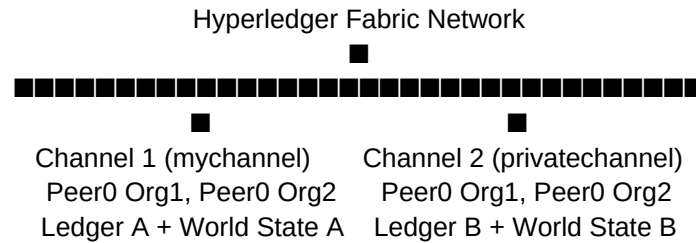
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile "$ORDERER_CA" -C mychannel -n basic -c '{"function":"CreateAsset","Args":["channelAsset1","Blue","10","Arun","1000"]}'
2026-09-08 13:31:51.622 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status 200 payload:{"ID":"channelAsset1","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1000}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query -C privatechannel -n basic -c '{"Args":["ReadAsset","channelAsset1"]}'
Error: endorsement failure during query. response: status:500 message:"The asset channelAsset1 does not exist"

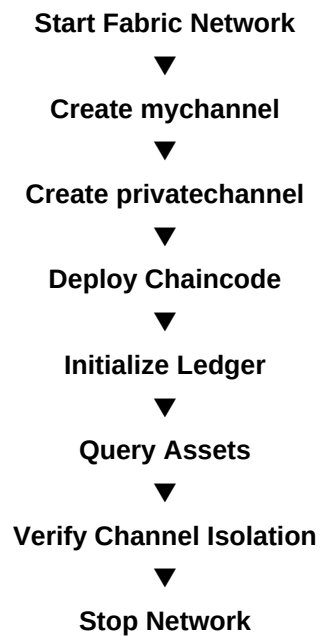
(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ ./network.sh down
```

Figure 3: InitLedger, CreateAsset on mychannel, channel isolation verification, and network shutdown

Architecture Diagram



Flow Diagram



Result

The Hyperledger Fabric network was successfully configured with multiple channels. A private channel was created, chaincode was deployed, and ledger transactions were executed successfully. The experiment demonstrated that channel members can access only the data belonging to their respective channels, ensuring privacy and confidentiality in a permissioned blockchain network.